

Teste de Software – Avaliação da Disciplina

Nome: Wady Jorge

Curso: Especialização em Tecnologia Java – UTFPR

Data: 05/06/2025

Especifique um conjunto de casos de teste para o seguinte programa

O programa recebe três valores inteiros que representam os lados de um triângulo. Com base nesses valores, ele determina se os lados formam um triângulo equilátero, isósceles ou escaleno.

Condição importante: Para que os lados formem um triângulo válido, a soma de quaisquer dois lados deve ser maior que o terceiro lado.

1. Implementação da classe Java

```
public class Triangulo {  
  
    private int lado1;  
    private int lado2;  
    private int lado3;  
  
    public Triangulo(int lado1, int lado2, int lado3) throws IllegalArgumentException {  
        // Validação inicial dos valores  
        if (lado1 <= 0 || lado2 <= 0 || lado3 <= 0) {  
            throw new IllegalArgumentException("Lados devem ser maiores que zero");  
        }  
  
        // Validação da soma dos lados  
        if (!validaSomaLados(lado1, lado2, lado3)) {  
            throw new IllegalArgumentException("A soma de dois lados deve ser maior que o  
terceiro");  
        }  
  
        this.lado1 = lado1;  
        this.lado2 = lado2;  
        this.lado3 = lado3;  
    }  
  
    private boolean validaSomaLados(int a, int b, int c) {  
        return (a + b > c) && (a + c > b) && (b + c > a);  
    }  
}
```

```

public String tipoTriangulo() {
    if (lado1 == lado2 && lado2 == lado3) {
        return "Equilatero";
    } else if (lado1 != lado2 && lado2 != lado3 && lado1 != lado3) {
        return "Escaleno";
    } else {
        return "Isosceles";
    }
}
}

```

2. Casos de teste JUnit

```

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class TrianguloTest {

    // --- Testes de triângulos válidos ---

    @Test
    void testEscaleno() throws Exception {
        Triangulo t = new Triangulo(3, 4, 5);
        assertEquals("Escaleno", t.tipoTriangulo());
    }

    @Test
    void testIsosceles() throws Exception {
        Triangulo t = new Triangulo(5, 5, 3);
        assertEquals("Isosceles", t.tipoTriangulo());
    }

    // Permutações para isósceles (testa todos os arranjos)
    @Test
    void testIsoscelesPermut1() throws Exception {
        Triangulo t = new Triangulo(5, 3, 5);
        assertEquals("Isosceles", t.tipoTriangulo());
    }

    @Test
    void testIsoscelesPermut2() throws Exception {
        Triangulo t = new Triangulo(3, 5, 5);
        assertEquals("Isosceles", t.tipoTriangulo());
    }

    @Test
    void testEquilatero() throws Exception {
        Triangulo t = new Triangulo(5, 5, 5);
    }
}

```

```

    assertEquals("Equilatero", t.tipoTriangulo());
}

// --- Testes de exceções para valores inválidos ---

// Lados com zero ou negativos (testa todos os lados separadamente)
@Test
void testLado1Zero() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(0, 5, 5));
}

@Test
void testLado2Zero() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(5, 0, 5));
}

@Test
void testLado3Zero() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(5, 5, 0));
}

@Test
void testLado1Negativo() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(-1, 5, 5));
}

@Test
void testLado2Negativo() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(5, -1, 5));
}

@Test
void testLado3Negativo() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(5, 5, -1));
}

@Test
void testTodosZero() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(0, 0, 0));
}

// --- Testes de exceção para soma inválida dos lados ---

// Soma de dois lados igual ao terceiro lado (não forma triângulo)
@Test
void testSomaIgual1() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(3, 7, 10)); // 3 + 7 =
10
}

```

```

@Test
void testSomaIgual2() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(7, 3, 10)); // 7 + 3 =
10
}

@Test
void testSomaIgual3() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(10, 3, 7)); // 3 + 7 =
10
}

// Soma de dois lados menor que o terceiro lado (não forma triângulo)
@Test
void testSomaMenor1() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(2, 3, 6)); // 2 + 3 < 6
}

@Test
void testSomaMenor2() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(3, 6, 2)); // 3 + 2 < 6
}

@Test
void testSomaMenor3() {
    assertThrows(IllegalArgumentException.class, () -> new Triangulo(6, 2, 3)); // 2 + 3 < 6
}
}

```

3. Percentual de cobertura de código

testesoftware-avaliacao01 > com.wadyjorge > Triangulo

Triangulo

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
Triangulo(int, int, int)		100%		100%	0	5	0	9	0	1
tipoTriangulo()		100%		100%	0	6	0	5	0	1
validaSomaLados(int, int, int)		100%		100%	0	4	0	1	0	1
Total	0 of 84	100%	0 of 24	100%	0	15	0	15	0	3